Name: Ronit Sahoo
Class: D20C
Roll No: 29
Batch: B

BLOCKCHAIN EXPERIMENT NO: 2

Aim - Create a Blockchain using Python.

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, a proof value obtained from Proof-of-Work, and the hash of the previous block, which together ensure the integrity and security of the chain.

In this program:

1. **Block Creation -** A block is generated using create_block() with a proof and the previous hash.
2. **Proof-of-Work (PoW) -** Implemented to mine a block by solving a cryptographic puzzle, i.e. finding a hash with the required leading zeros.
3. **Hashing -** The SHA-256 algorithm is used to ensure data immutability.
4. **Validation -** is_chain_valid() checks that every block correctly references the previous hash and satisfies the PoW condition.
5. **Flask Web API -** The routes /mine_block, /get_chain and /is_valid allow mining, fetching the chain and verifying blockchain integrity through a browser or an API client such as Postman.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Tasks Performed:

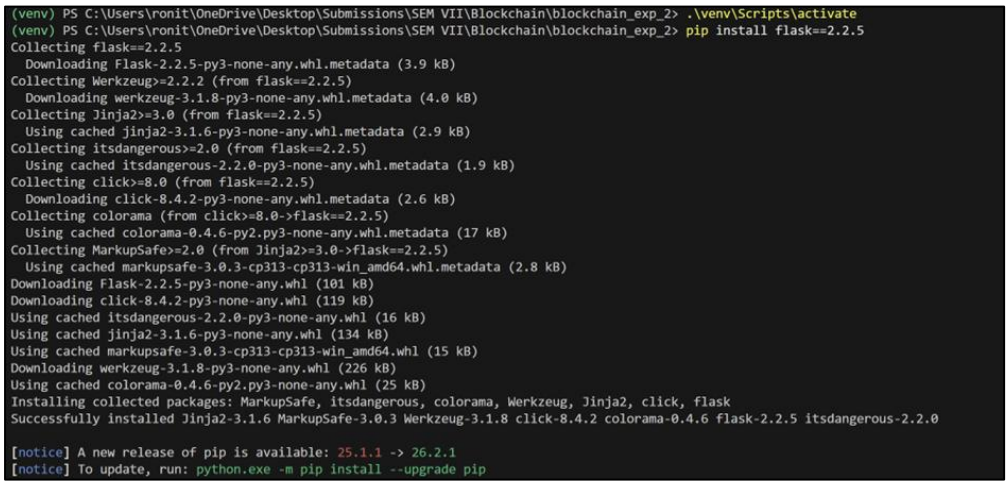
1. **Importing the Required Libraries:** datetime is used to timestamp each block, hashlib to generate SHA-256 hashes, json to encode blocks before hashing, and Flask to run the blockchain API server.
2. **Building the Blockchain Class:** Implemented the core logic to create blocks, store the chain, perform Proof-of-Work and validate the integrity of the chain.
3. **Exposing a Flask Web API:** Created the endpoints /mine_block to mine a new block, /get_chain to fetch the complete blockchain and /is_valid to verify the blockchain.
4. **Testing the Endpoints:** Ran the Flask server on 127.0.0.1:5000 and tested every route using Postman to observe the genesis block, newly mined blocks and the validity of the chain.

Program & Output:

1. Setting Up the Environment and Installing Flask

```
.\venv\Scripts\activate
pip install flask==2.2.5
```

Output:



2. Blockchain Implementation (blockchain.py)

```
import datetime
import hashlib
import json
from flask import Flask, jsonify

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []
        # Create Genesis Block
        self.create_block(proof=1,
previous_hash='0')

    def create_block(self, proof,
previous_hash, transactions=None):
        if transactions is None:
            transactions = []
        block = {
            'index': len(self.chain) + 1,
            'timestamp':
str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash':
previous_hash,
            'transactions': transactions
        }
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def create_Transactions(self):
        # Mine only when the transaction
list is not empty
        if self.transactions:
            previous_block =
self.get_previous_block()
            previous_proof =
previous_block['proof']
            # Find the golden nonce
            proof =
self.proof_of_work(previous_proof)
            # Get the hash of the
previous block
            previous_hash =
self.hash(previous_block)
            # Remove transactions from
the pending list before mining
            transactions_to_mine =
self.transactions.copy()
            self.transactions = []
            # Create a new block
containing the transactions
            block =
self.create_block(proof, previous_hash,
transactions_to_mine)
            return block
        # No transactions -> do not mine
        return None

    def proof_of_work(self,
previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation =
hashlib.sha256(
                str(new_proof**2 -
previous_proof**2).encode()
            ).hexdigest()
            # Cryptographic puzzle: the
hash must start with 000
            if hash_operation[:3] ==
'000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block,
sort_keys=True).encode()
        return
hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            # Check the previous hash
            if block['previous_hash'] !=
self.hash(previous_block):
                return False
            # Check the Proof-of-Work
            previous_proof =
previous_block['proof']
            proof = block['proof']
            hash_operation =
hashlib.sha256(
                str(proof**2 -
previous_proof**2).encode()
            ).hexdigest()
            if hash_operation[:3] !=
'000':
                return False
            previous_block = block
            block_index += 1
        return True

# Flask Application
app = Flask(__name__)
blockchain = Blockchain()

# Mine a Block
@app.route('/mine_block',
methods=['GET'])
def mine_block():
```

```
    previous_block =
blockchain.get_previous_block()
    previous_proof =
previous_block['proof']
    proof =
blockchain.proof_of_work(previous_proof)
    previous_hash =
blockchain.hash(previous_block)
    block =
blockchain.create_block(proof,
previous_hash, blockchain.transactions)
    # Remove the transactions after
adding them to the block
    blockchain.transactions = []
    response = {
        'message': 'Congratulations, you
just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash':
block['previous_hash'],
        'transactions':
block['transactions']
    }
    return jsonify(response), 200
```

```
# Get the full Blockchain
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

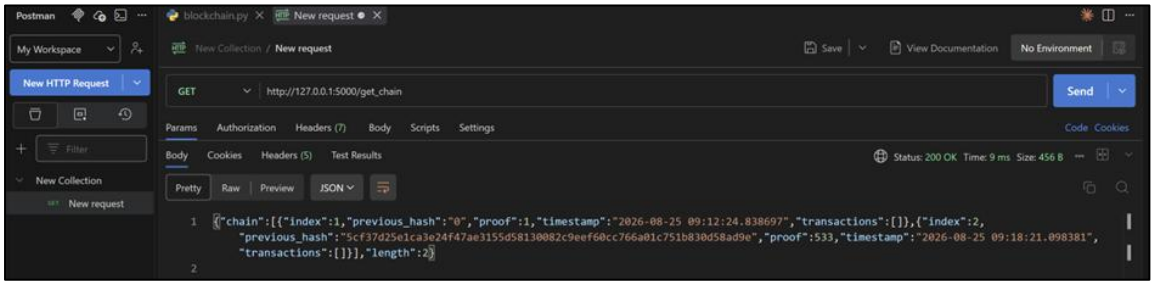
# Check Blockchain Validity
@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid =
blockchain.is_chain_valid(blockchain.chai
n)
    if valid:
        response = {'message': 'All good.
The Blockchain is valid.'}
    else:
        response = {'message': 'The
Blockchain is not valid.'}
    return jsonify(response), 200

# Run the Flask Server
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

3. Fetching the Blockchain - GET /get_chain

The request returns the complete chain along with its length. At this point the chain holds the genesis block (proof = 1, previous_hash = 0) and one mined block, so the length is 2.

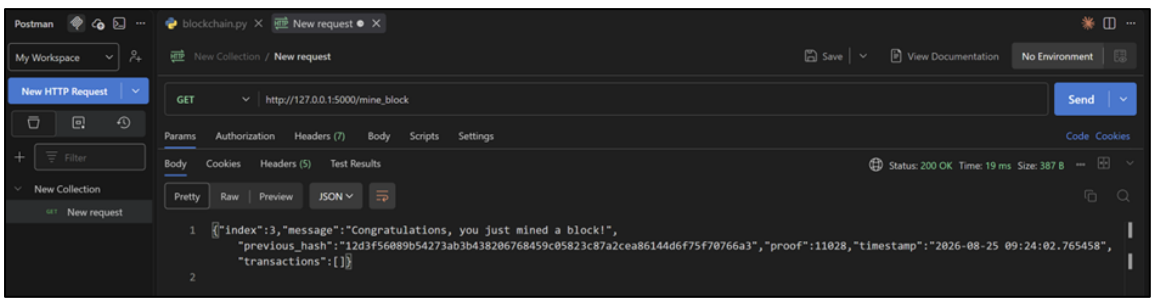
Output:



4. Mining a New Block - GET /mine_block

Calling this route runs the Proof-of-Work algorithm until a valid nonce is found. A new block with index 3 is appended to the chain, linked to the previous block through its hash.

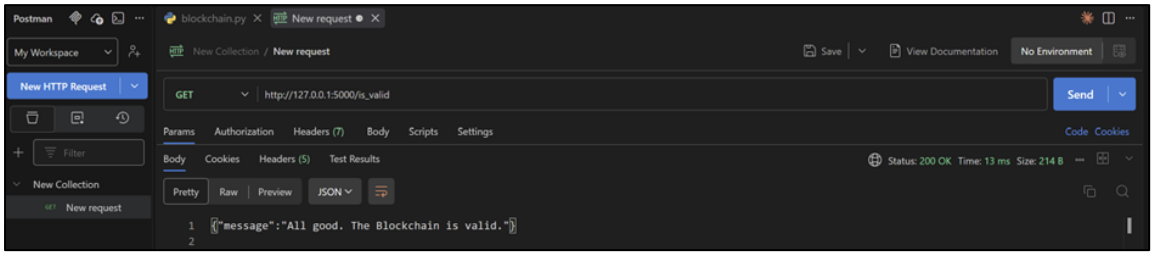
Output:



5. Verifying the Blockchain - GET /is_valid

Every block is re-hashed and compared with the previous_hash stored in the next block, and each proof is re-verified against the puzzle condition. The chain passes both checks.

Output:



Conclusion:

In this experiment a basic blockchain was successfully created using Python and Flask, supporting block mining, chain retrieval and validity checking through a REST API. The following points were observed:

1. **Proof-of-Work** makes block creation computationally intensive, which is what secures the network against tampering.
2. **SHA-256 hashing** links every block to its predecessor, so any change to a past block invalidates all the blocks that follow it.
3. **The validation routine** confirms both the hash linkage and the difficulty condition, proving the immutability of the chain.
4. **The Flask API** makes the blockchain accessible over HTTP, allowing the chain to be mined, inspected and verified from Postman.

Overall, the implementation demonstrates the core blockchain principles of decentralization, immutability and consensus in a simplified environment, and provides a strong foundation for building more advanced blockchain systems with transactions and multiple nodes.